Jakub Skibicki 20266112

AST project - individual report

My contribution to the project

I built the foundation that the rest of the tool depends on: the benchmark function definitions, safe input domains, input sampling, mutant generation, and the metrics used to score discovered metamorphic relations. I created the core function registry in amr/functions.py. It contains the eight functions used in the project: abs, asinh, atan, cos, log1p, log10, sin, and tan. I also defined the domains for each function. This was an important part of the project because the search only works if it samples valid and numerically stable inputs. For example, log10 must stay above zero, log1p must stay above -1, and tan has to avoid values close to its asymptotes. These domain choices made the later PSO search and validation steps much more stable. I also implemented input sampling in amr/data.py. The sampler draws values from the valid domain of a selected function and uses seeds so that experiments are reproducible. This made it possible to run the same tests and evaluations multiple times without getting inconsistent results caused by random sampling. Another major part of my work was the mutant system in amr/mutants.py. I defined the mutation operators used to create faulty versions of the original functions, including adding or multiplying the output by a constant, shifting or scaling the input, replacing the output with a constant, and negating the output. These mutants are what allow us to check whether a discovered metamorphic relation is actually useful for finding bugs, not just mathematically valid on the original function. I also wrote the scoring logic in amr/metrics.py. This module checks whether a mutant violates a discovered relation and calculates the kill rate, meaning the fraction of mutants detected by at least one relation. I also added the precision metric for accepted relations. These metrics define how we evaluate the effectiveness of the whole tool, so this part connects the relation discovery process with the final experimental results. Finally, I added tests for my modules in the tests/ folder. The tests check that all benchmark functions are registered correctly, that domains are valid, that sampling is reproducible, that mutant generation works as expected, and that the kill-rate calculation correctly distinguishes killed and surviving mutants. I also helped set up the project structure around the amr and test folders so that the rest of the implementation could build on these modules and wrote the first 2 sections of the final report.

Peer evaluation

Ada (10/10)

Ada implemented the LLM reconnaissance module and the shared project interfaces. Her part asks a local language model for high-level hints about each function, such as symmetry, periodicity, domain, and range, while keeping the model output advisory rather than authoritative. She also added caching, strict JSON handling, and tests with mocked model calls. Along with Hugo, she presented during midterm and final presentation.

Hugo (10/10)

Hugo implemented the main search mechanism. He wrote the PSO algorithm, the conversion from function profiles to parameter bounds, and the fitness and validation logic. His work made sure that the tool did not accept trivial or meaningless relations and that discovered relations were checked on fresh inputs before being reported. Along with Ada, he presented during midterm and final presentation.

Yeskendir (10/10)

Yeskendir connected the separate modules into one runnable tool. He implemented the command-line interface in amr/cli.pyand the main pipeline in amr/run.py, including restarts, result writing, summary generation, and rerun handling. He also prepared the Apache Commons Math 2.2 benchmark, comparison scripts, reference data for MRI and AutoMR, related tests, and wrote the “Solution” section of the report.Yeskendir connected the separate modules into one runnable tool. He implemented the command-line interface in amr/cli.pyand the main pipeline in amr/run.py, including restarts, result writing, summary generation, and rerun handling. He also prepared the Apache Commons Math 2.2 benchmark, comparison scripts, reference data for MRI and AutoMR and related tests.